NOMAD: A Device-Agnostic, Verifier-in-the-Loop Agentic Kernel Compiler

CS 521 Final Project Proposal — Project 5 (Agentic AI Kernel Compiler), subsuming Projects 1–3

Hemank Kohli
University of Illinois Urbana–Champaign
USA
hkohli2@illinois.edu

Omer Sajid
University of Illinois Urbana–Champaign
USA
mosajid2@illinois.edu

1 Problem and motivation

Agentic kernel generation is the live frontier (KernelBench → CAKE → TritorX → AWS's Trainium NKI agent), yet three gaps remain. (1) **Single-vendor:** CAKE is NVIDIA-only, TritorX targets one ASIC, and AWS's agent needs a Trainium device in the loop; none retargets from a declarative hardware description. (2) **Kernel-level, not graph-level:** fusion (P3), data layout (P1), and rewrite synthesis (P2), the optimizations that move end-to-end performance, sit outside agent action spaces. (3) **Correctness is empirical:** kernels are admitted by `np.allclose` or OpInfo tests, audits (KernelBench-Verified, KernelGuard) find many test-passing kernels broken, and formal checkers (Volta, ProofWright) are offline and NVIDIA-specific.

Question. An agent's *decisions* (fusion boundaries, layouts, sync edges, staging) are device-independent; only their *realization* is device-specific. Can one agent + verifier stack, working on a portable IR parameterized by a device spec, *soundly gate* a fused attention block for NVIDIA GPUs and Trainium, with device-free feedback and measured end-to-end wins?

2 What we will implement

(1) DeviceSpec. One YAML per device (compute roles, memory tiers, copy engines, sync primitives, capability flags) parameterizes the IR, checks, and cost model; v0: `sm86` (our RTX 3070), `trn2`.

(2) Portable Kernel IR (PKIR). Tile-level dataflow (memory-tier buffers, worker roles, staged pipelines, explicit sync edges, affine index maps); device instructions enter only as capability-gated ops with *semantic contracts*. Template lowerings to CUDA and NKI.

(3) Agent + harness. An off-the-shelf coding agent behind a fixed CLI edits fusion plans (DNNFusion-style), layouts (Einsum-Tree-style), gated rewrites (TASO-style), and PKIR schedules. Ordered gates (static checks → verification → lowering → differential tests → timing) return localized diagnostics; a spec-driven cost model stands in for hardware.

(4) Verifier. Sound for static shapes, affine indexing, and structured sync: memory safety, race/deadlock freedom via sync edges, and equivalence of PKIR regions to the reference (symbolic tile dataflow). Backends are differentially tested.

Scope. One fused attention block (RMSNorm → QKV → SDPA with online softmax → out-proj, fp16); RTX 3070 for all measurements; Trainium2 via NKI simulator + NEFF compile (on-device is a bonus).

3 Evaluation

(1) Block latency vs. PyTorch eager, `torch.compile`, and SDPA. (2) Gate study: ≈10 injected bugs (dropped barrier, skipped K-tile, OOB tile, …) caught by tests vs. verifier vs. Compute Sanitizer; agent with vs. without gates. (3) Portability: only the DeviceSpec changes between CUDA and NKI; retarget cost. (4) Ablation: kernel-only vs. graph-level agent.

4 Plan and risks

HK (Hemank): CUDA path, agent, GPU experiments; OS (Omer): DeviceSpec/PKIR, verifier, NKI path; integration points I1–I4 are pair-programmed. Figure 1 gives the plan: 196 person-hours (HK 99, OS 97), ≈18 h/person/week over ≈5.5 working weeks, with Thanksgiving week as buffer. **Risks.** If NKI lowering slips, restrict Trainium to softmax; if the equivalence core slips, fall back to static + differential checks + Volta on CUDA. Never cut: GPU end-to-end results and the bug-injection study. Unbudgeted stretch goals: Volta gate on final CUDA kernels, on-device NKI runs. *AI-assisted coding will be acknowledged; interpretation of results is ours.*

Task	Owner	Hours
GPU env, PyTorch reference block + baselines, differential-test harness	HK	12
DeviceSpec v0 (`sm86`, `trn2`) + contract schema; NKI env smoke test	OS	12
PKIR spec v0 + interfaces; integration points I1–I4 (I2 = progress report)	both	15+15
PKIR→CUDA lowering (tiles, copies, mma, reductions, exp)	HK	14
PKIR builder + type/capability checker	OS	14
Graph-level actions v0: fusion catalog + layout set	HK	12
Verifier v0: OOB/memory safety + race/deadlock analyses	OS	12
Agent driver around harness CLI; token accounting; pilot runs	HK	8
Semantic-equivalence core v0 (matmul/elementwise/reduction)	OS	12
Agent schedule runs + GPU tuning; bug-injection suite (≈10 mutants)	HK	18
PKIR→NKI lowering (softmax, RMSNorm) + portability run (spec swap)	OS	12
Ablations: kernel-only vs. graph-level agent, ± gates, two device specs	both	8+8
Final report (ASPLOS acmart, 5 pp.) + artifact README/scripts	both	12+12
Total (HK 99 h, OS 97 h; ≈18 h/person/week)		**196**

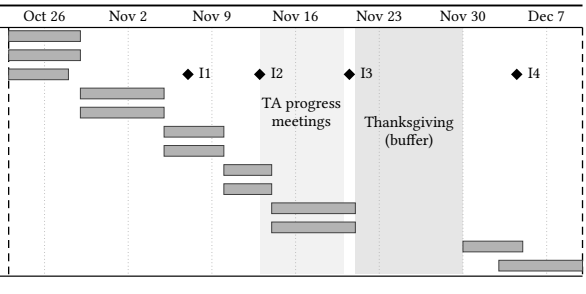


Figure 1: Tasks and timeline (HK = Hemank, OS = Omer; ◆ = integration point; dashed = proposal and final deadlines).